Zephyr Project

IT-Schulungen.com
Technology & Transformation

Gemeinsamer Zustand und Race Conditions

Synchronisation, Ownership und Fehlersuche

2.08

Verlorenes Update: ++ ist keine Einzeloperation

READ · MODIFY · WRITE

```
uint32_t local = shared_count;
local = local + 1U;
shared_count = local;
```

01 Lesen

Jeder Thread lädt denselben Ausgangswert aus dem Speicher.

02 Ändern

Die Berechnung erfolgt zunächst nur in einer lokalen Kopie.

03 Schreiben

Der spätere Schreibzugriff überschreibt die Aktualisierung des anderen Threads.

Zwei Threads, ein verlorenes Update

01	`Ausgang`	shared_count = 41
02	`Thread A`	liest 41 · berechnet 42 · schreibt 42
03	`Thread B`	liest 41 · berechnet 42 · schreibt 42

Erwartet: 43 · Beobachtet: 42 — keine einzelne Zeile ist „falsch“, die Verschachtelung ist es.

Drei Fehlerbilder sauber trennen

BEGRIFFE

Nicht jeder Timingfehler ist automatisch ein Datenrennen.

Race Condition	Ergebnis hängt von der zeitlichen Verschachtelung ab
Datenrennen	Ungeschützte, überlappende Zugriffe auf dieselben Daten
Reihenfolgefehler	Zulässige Einzeloperationen passieren logisch zu früh oder zu spät
Typisches Symptom	Sporadisch falscher Zähler — selten und lastabhängig

Reproduzierbarkeit widerlegt keine Race Condition: anderes Timing kann den Fehler nur verdecken.

Kritischer Abschnitt: minimal und eindeutig

MUTEX

```
k_mutex_lock(&lock, K_FOREVER);
shared_count++;
k_mutex_unlock(&lock);
```

01 Invariante

Zwischen Lesen und Schreiben darf kein anderer Thread denselben Zustand verändern.

02 Klein halten

Keine unnötige Arbeit, kein Logging und kein blockierender Aufruf unter dem Lock.

Mutex schützt Besitz und kritischen Abschnitt – nicht automatisch jede logische Reihenfolge.

Werkzeugwahl: drei unterschiedliche Rollen

MUTEX

Besitzschutz

Kritischer Abschnitt im Thread-Kontext

Genau ein Besitzer entsperrt

SEMAPHOR

Signal oder Zähler

Kein Besitz

Ereignisse oder Ressourcen zählen

ATOMIC

Kleine Operation

Einzelne Zähler oder Flags ändern

Keine zusammengesetzte Invariante

Werkzeug nach Semantik wählen: schützen, signalisieren oder atomar ändern.

Prioritätsinversion und Deadlock erkennen

Niedrige Priorität hält Mutex	Ein hochpriorer Thread wartet auf die Freigabe
Mittlere Priorität läuft	Die Wartezeit des hochprioren Threads wächst
Prioritätsvererbung	Der Mutex-Besitzer wird temporär höher priorisiert
Zyklische Abhängigkeit	Thread A wartet auf B — Thread B wartet auf A
Gegenmaßnahmen	Feste Sperrreihenfolge, kurze Abschnitte, klare Fehlerpfade

Prioritätsvererbung mildert Inversion; sie ersetzt keine konsistente Sperrstrategie.

Kurzcheck: Gemeinsamer Zustand

ERKENNEN

Woran liegt es?

++ ist Read–Modify–Write

Fehler ist timingabhängig

Erwarteter Endwert fehlt

Reproduzierbarkeit schwankt

SCHÜTZEN

Passendes Werkzeug

Mutex für Besitzschutz

Semaphore für Signale

Atomic für Einzeloperationen

kritischen Abschnitt minimieren

VERMEIDEN

Entwurf prüfen

Sperrreihenfolge festlegen

blockierende Arbeit vermeiden

Ownership dokumentieren

Daten lieber übertragen

Transferfrage: Welches Werkzeug schützt einen gemeinsam veränderten mehrteiligen Zustand – und warum?